

Chapter 44: Segment Tree & Fenwick Tree

Personal Notes

Prefix sums (Ch 40) gave you $O(1)$ range queries — but ONLY if the array never changes. The moment values start updating, prefix sums fall apart: a single update forces you to rebuild the prefix array in $O(n)$, and a thousand updates become $O(n \times \text{updates})$.

Segment trees and Fenwick trees fix this. Both handle range queries AND point updates in $O(\log n)$ each — a huge improvement when both operations happen frequently. Segment trees are more general (any associative operation: sum, min, max, GCD). Fenwick trees (also called Binary Indexed Trees, or BIT) are smaller and faster for the common case of range SUMS. This chapter covers both, plus when to use which.

1. Why Not Just Use Prefix Sums?

Operation	Prefix Sums	Segment / Fenwick Tree
Build	$O(n)$	$O(n)$
Range sum query	$O(1)$	$O(\log n)$
Point update	$O(n)$ — rebuild	$O(\log n)$
Many updates + queries mixed	$O(n \times Q)$	$O(\log n \times Q)$

The trade-off: Prefix sums win when the array is FIXED — $O(1)$ queries beat $O(\log n)$. Segment/Fenwick trees win when updates and queries are BOTH FREQUENT. The break-even is roughly "as soon as updates start happening," segment trees are the answer.

2. The Segment Tree Idea

A SEGMENT TREE is a binary tree where each node represents a RANGE of the array, and stores an aggregate (sum, min, max, whatever) over that range. The root covers $[0, n-1]$. Each node has two children covering the LEFT and RIGHT halves of its range. Leaves cover single elements.

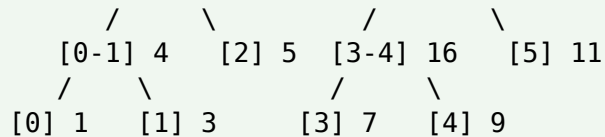
```
arr = [1, 3, 5, 7, 9, 11]
```

Segment tree (values are SUMS over each range):

```

      [0-5] 36
     /    \
  [0-2] 9   [3-5] 27

```



Query for `sum(1..4)`:

- `[0-1]` contributes only `[1]` 3
- `[2]` contributes fully: 5
- `[3-4]` contributes fully: 16
- Total: $3 + 5 + 16 = 24$ ✓ ($= 3 + 5 + 7 + 9$)

Why It's Fast

- Query: $O(\log n)$ — visit at most 4 nodes per level, $\log n$ levels
- Update: $O(\log n)$ — update leaf, then bubble up to root
- Space: $O(4n)$ — a safe upper bound for the array-based storage

3. Array-Backed Storage

Just like heaps (Ch 34), segment trees are typically stored in a flat array. If node index is i :

- Left child = $2*i + 1$ (or $2*i$ in 1-indexed schemes)
- Right child = $2*i + 2$ (or $2*i + 1$)
- Parent = $(i - 1) / 2$

```
// For safety, allocate 4n slots:
int[] tree = new int[4 * n];
```

Why 4n and not 2n: A perfect binary tree with n leaves has $2n-1$ nodes. But when n isn't a power of 2, the array-based representation leaves gaps. $4n$ is a safe upper bound that always works — the extra memory is cheap compared to debugging index-out-of-bounds errors.

4. Building a Segment Tree — Full Code

This is a range-SUM segment tree. Change one line (the combine step) to make it range-MIN, range-MAX, range-GCD, etc.

```
class SegmentTree {
    int[] tree;
    int n;

    public SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, 0, n - 1);
    }
}
```

```

private void build(int[] arr, int node, int lo, int hi) {
    if (lo == hi) {
        tree[node] = arr[lo];
        return;
    }
    int mid = lo + (hi - lo) / 2;
    build(arr, 2*node + 1, lo, mid);
    build(arr, 2*node + 2, mid + 1, hi);
    tree[node] = tree[2*node + 1] + tree[2*node + 2];    // combine
}

public int query(int l, int r) {
    return query(0, 0, n - 1, l, r);
}

private int query(int node, int lo, int hi, int l, int r) {
    if (r < lo || hi < l) return 0;                // outside range
    if (l <= lo && hi <= r) return tree[node];    // fully inside
    int mid = lo + (hi - lo) / 2;
    return query(2*node + 1, lo, mid, l, r)
        + query(2*node + 2, mid + 1, hi, l, r);
}

public void update(int i, int val) {
    update(0, 0, n - 1, i, val);
}

private void update(int node, int lo, int hi, int i, int val) {
    if (lo == hi) { tree[node] = val; return; }
    int mid = lo + (hi - lo) / 2;
    if (i <= mid) update(2*node + 1, lo, mid, i, val);
    else update(2*node + 2, mid + 1, hi, i, val);
    tree[node] = tree[2*node + 1] + tree[2*node + 2];
}
}

```

The 'combine' line is what you customize: For range-MIN, use Math.min. For range-MAX, use Math.max. For range-GCD, use gcd(). The rest of the code is identical. That's the elegance of segment trees — one skeleton, many operations.

5. Query Walkthrough

Understanding how query works recursively is key. Every recursive call handles one of three cases:

1. NO OVERLAP with the query range → return the identity (0 for sum, INF for min, etc.)
2. FULLY INSIDE the query range → return the stored aggregate
3. PARTIAL OVERLAP → recurse into both children and combine

```
Query sum(1..4) on arr = [1, 3, 5, 7, 9, 11]:
```

```
node [0-5]: partial overlap → recurse both children
  node [0-2]: partial overlap → recurse both children
    node [0-1]: partial overlap → recurse both children
      node [0]: no overlap → return 0
      node [1]: fully inside → return 3
      combined: 3
    node [2]: fully inside → return 5
    combined: 3 + 5 = 8
  node [3-5]: partial overlap → recurse both children
    node [3-4]: fully inside → return 16
    node [5]: no overlap → return 0
    combined: 16 + 0 = 16
  combined: 8 + 16 = 24 ✓
```

6. The Fenwick Tree — A Simpler Alternative

For the specific case of range SUMS (or any INVERTIBLE operation — you can "undo" it), the FENWICK TREE (Binary Indexed Tree, or BIT) is a smaller, simpler alternative. Same $O(\log n)$ query and update times, but half the code and less memory.

The Idea

Instead of a full binary tree, use an array where each index i stores the sum of a specific POWER-OF-2 range ending at i . The trick uses $n \& -n$ (the "lowest set bit" from Chapter 38) to navigate.

```
arr:      [1, 2, 3, 4, 5, 6, 7, 8]      (1-indexed for Fenwick)
BIT[i]:    [1, 3, 3, 10, 5, 11, 7, 36]
```

```
BIT[1] = arr[1]           = 1
BIT[2] = arr[1..2]         = 3
BIT[3] = arr[3]           = 3
BIT[4] = arr[1..4]         = 10
BIT[5] = arr[5]           = 5
BIT[6] = arr[5..6]         = 11
BIT[7] = arr[7]           = 7
BIT[8] = arr[1..8]         = 36
```

Each index's range is determined by its lowest set bit.

E.g., $i=6 = 0110$, lowest set bit = 010 (2), so BIT[6] covers a range of size 2 ending at 6.

Fenwick Tree Code

```
class FenwickTree {
    int[] bit;
    int n;
```

```

public FenwickTree(int n) {
    this.n = n;
    bit = new int[n + 1];    // 1-indexed
}

// Add 'delta' at position i (1-indexed)
public void update(int i, int delta) {
    for (; i <= n; i += i & -i) {
        bit[i] += delta;
    }
}

// Prefix sum arr[1..i]
public int query(int i) {
    int sum = 0;
    for (; i > 0; i -= i & -i) {
        sum += bit[i];
    }
    return sum;
}

// Range sum arr[l..r]
public int rangeQuery(int l, int r) {
    return query(r) - query(l - 1);
}
}

```

The magic of $i \& -i$: $i \& -i$ isolates the LOWEST SET BIT of i . In update, adding it moves you UP the tree. In query, subtracting it moves you to the previous non-overlapping range. This bit trick is why Fenwick trees are so compact — the tree structure is IMPLICIT in the bit patterns of the indices.

7. Segment Tree vs Fenwick Tree — Which to Use?

Aspect	Segment Tree	Fenwick Tree
Operations supported	Any associative (sum, min, max, GCD, etc.)	Only invertible (sum, XOR)
Code size	~40 lines	~15 lines
Memory	$O(4n)$	$O(n)$
Constant factor	Larger — recursive	Smaller — iterative + bit tricks
Range MIN / MAX	Yes — natural fit	Not directly (no undo)
Point updates	$O(\log n)$	$O(\log n)$
Range updates	Needs lazy propagation	Difference array trick works for

Aspect	Segment Tree	Fenwick Tree
		some

One-line rule of thumb: Range-SUM problems → Fenwick tree (simpler, faster). Range-MIN / range-MAX / anything non-invertible → Segment tree. When in doubt, learn segment trees first — they're more general and worth writing correctly once for muscle memory.

8. Classic Problem 1 — Range Sum Query (Mutable)

LeetCode 307. Given an array that gets updated repeatedly, answer range sum queries efficiently. THE textbook segment/Fenwick tree problem.

```
class NumArray {
    FenwickTree bit;
    int[] nums;

    public NumArray(int[] nums) {
        this.nums = nums.clone();
        bit = new FenwickTree(nums.length);
        for (int i = 0; i < nums.length; i++) {
            bit.update(i + 1, nums[i]);
        }
    }

    public void update(int index, int val) {
        bit.update(index + 1, val - nums[index]); // apply delta
        nums[index] = val;
    }

    public int sumRange(int left, int right) {
        return bit.rangeQuery(left + 1, right + 1);
    }
}

// Time: O(log n) per operation
```

The delta trick: update(i, val) needs to CHANGE the value at index i to val. Since Fenwick trees track ADDITIONS, compute the delta (val - old_value) and apply that. This is the standard pattern for turning "set to value" into "add delta."

9. Classic Problem 2 — Range Sum Query 2D (Mutable)

LeetCode 308. 2D version of the above. Use a 2D Fenwick tree — same idea, two nested update/query loops using i & -i.

```

class NumMatrix {
    int[][] bit;
    int[][] mat;
    int rows, cols;

    public NumMatrix(int[][] matrix) {
        rows = matrix.length; cols = matrix[0].length;
        bit = new int[rows + 1][cols + 1];
        mat = new int[rows][cols];
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                update(i, j, matrix[i][j]);
    }

    public void update(int r, int c, int val) {
        int delta = val - mat[r][c];
        mat[r][c] = val;
        for (int i = r + 1; i <= rows; i += i & -i)
            for (int j = c + 1; j <= cols; j += j & -j)
                bit[i][j] += delta;
    }

    private int query(int r, int c) {
        int sum = 0;
        for (int i = r; i > 0; i -= i & -i)
            for (int j = c; j > 0; j -= j & -j)
                sum += bit[i][j];
        return sum;
    }

    public int sumRegion(int r1, int c1, int r2, int c2) {
        return query(r2 + 1, c2 + 1) - query(r1, c2 + 1)
            - query(r2 + 1, c1) + query(r1, c1);
    }
}

```

Same inclusion-exclusion formula as the 2D prefix sums from Ch 40 — but now updates are $O(\log^2 n)$ instead of $O(n^2)$.

10. Classic Problem 3 — Count Smaller Numbers After Self

LeetCode 315. For each element, count how many elements to its RIGHT are smaller. Beautiful application of Fenwick trees on VALUE space.

The Trick

Scan the array from RIGHT to LEFT. For each value, ask: "how many values LESS than me have I already seen?" That's a prefix sum query on a Fenwick tree indexed by VALUE.

```

public List<Integer> countSmaller(int[] nums) {

```

```

// Coordinate compression – map each unique value to an index
TreeSet<Integer> sorted = new TreeSet<>();
for (int n : nums) sorted.add(n);
Map<Integer, Integer> rank = new HashMap<>();
int r = 1;
for (int v : sorted) rank.put(v, r++);

FenwickTree bit = new FenwickTree(sorted.size());
Integer[] result = new Integer[nums.length];

for (int i = nums.length - 1; i >= 0; i--) {
    int idx = rank.get(nums[i]);
    result[i] = bit.query(idx - 1); // count of smaller-ranked so
far
    bit.update(idx, 1); // mark this value as seen
}
return Arrays.asList(result);
}

// Time: O(n log n) Space: O(n)

```

Coordinate compression is a companion trick: When values are huge (like 10^9) but there are only n distinct ones ($n \leq 10^5$), map each value to a small rank so the Fenwick tree stays reasonably-sized. This trick shows up in many advanced Fenwick tree problems.

11. Classic Problem 4 — Range Minimum Query

Given an array, answer range MIN queries with point updates. Fenwick trees don't work (no undo for min), so we use a Segment Tree with the combine step changed to `Math.min`.

```

class MinSegmentTree {
    int[] tree;
    int n;

    public MinSegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        Arrays.fill(tree, Integer.MAX_VALUE);
        build(arr, 0, 0, n - 1);
    }

    private void build(int[] arr, int node, int lo, int hi) {
        if (lo == hi) { tree[node] = arr[lo]; return; }
        int mid = lo + (hi - lo) / 2;
        build(arr, 2*node + 1, lo, mid);
        build(arr, 2*node + 2, mid + 1, hi);
        tree[node] = Math.min(tree[2*node + 1], tree[2*node + 2]);
    }
}

```



```

public int queryMin(int l, int r) {
    return query(0, 0, n - 1, l, r);
}

private int query(int node, int lo, int hi, int l, int r) {
    if (r < lo || hi < l) return Integer.MAX_VALUE;
    if (l <= lo && hi <= r) return tree[node];
    int mid = lo + (hi - lo) / 2;
    return Math.min(query(2*node + 1, lo, mid, l, r),
                    query(2*node + 2, mid + 1, hi, l, r));
}

public void update(int i, int val) {
    update(0, 0, n - 1, i, val);
}

private void update(int node, int lo, int hi, int i, int val) {
    if (lo == hi) { tree[node] = val; return; }
    int mid = lo + (hi - lo) / 2;
    if (i <= mid) update(2*node + 1, lo, mid, i, val);
    else          update(2*node + 2, mid + 1, hi, i, val);
    tree[node] = Math.min(tree[2*node + 1], tree[2*node + 2]);
}
}

```

Notice the pattern: identical structure to the sum tree, but every combine step uses `Math.min` and the identity is `Integer.MAX_VALUE` (instead of 0 for sum).

12. Lazy Propagation — Range Updates (Advanced)

So far, updates have been POINT updates (change one element). What about RANGE updates — "add 5 to every element from index 3 to 7"? Naive: $O(\text{range} \times \log n)$ per update. Better: LAZY PROPAGATION, which makes range updates $O(\log n)$ too.

The Idea

Instead of updating every leaf in the range, mark the NODE that fully covers the range with a pending update. When we later need to descend into that node, PROPAGATE the pending update to its children. This defers work until it's actually needed.

```

class LazySegmentTree {
    long[] tree;
    long[] lazy;
    int n;

    public LazySegmentTree(int[] arr) {
        n = arr.length;
        tree = new long[4 * n];
    }
}

```

```

        lazy = new long[4 * n];
        build(arr, 0, 0, n - 1);
    }

    private void build(int[] arr, int node, int lo, int hi) {
        if (lo == hi) { tree[node] = arr[lo]; return; }
        int mid = lo + (hi - lo) / 2;
        build(arr, 2*node + 1, lo, mid);
        build(arr, 2*node + 2, mid + 1, hi);
        tree[node] = tree[2*node + 1] + tree[2*node + 2];
    }

    private void push(int node, int lo, int hi) {
        if (lazy[node] != 0) {
            int mid = lo + (hi - lo) / 2;
            int L = 2*node + 1, R = 2*node + 2;
            tree[L] += lazy[node] * (mid - lo + 1);
            tree[R] += lazy[node] * (hi - mid);
            lazy[L] += lazy[node];
            lazy[R] += lazy[node];
            lazy[node] = 0;
        }
    }

    public void rangeAdd(int l, int r, long val) {
        rangeAdd(0, 0, n - 1, l, r, val);
    }

    private void rangeAdd(int node, int lo, int hi, int l, int r, long
val) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) {
            tree[node] += val * (hi - lo + 1);
            lazy[node] += val;
            return;
        }
        push(node, lo, hi);
        int mid = lo + (hi - lo) / 2;
        rangeAdd(2*node + 1, lo, mid, l, r, val);
        rangeAdd(2*node + 2, mid + 1, hi, l, r, val);
        tree[node] = tree[2*node + 1] + tree[2*node + 2];
    }

    public long query(int l, int r) {
        return query(0, 0, n - 1, l, r);
    }

    private long query(int node, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return 0;

```

```

    if (l <= lo && hi <= r) return tree[node];
    push(node, lo, hi);
    int mid = lo + (hi - lo) / 2;
    return query(2*node + 1, lo, mid, l, r)
           + query(2*node + 2, mid + 1, hi, l, r);
}
}

```

Lazy propagation is genuinely tricky: Get the details wrong and it silently produces incorrect answers. Common bugs: forgetting to multiply lazy value by segment length, applying push in the wrong order, forgetting to zero out lazy after propagating. If you must implement it in an interview, use a test with 2-3 hand-traced range updates and queries.

13. When to Reach for These Structures

If you need to...	Use
Range sum + point updates	Fenwick tree (BIT)
Range min / max + point updates	Segment tree with min/max combine
Range GCD / LCM queries	Segment tree
Range update + point query	Fenwick tree of DIFFERENCES
Range update + range query	Segment tree with LAZY propagation
2D range sum + point updates	2D Fenwick tree
Count elements less than X in a prefix	Fenwick tree indexed by VALUE (coord compress)
Count inversions in an array	Fenwick tree — process right-to-left
K-th smallest with dynamic array	Segment tree / policy tree
Persistent range queries (history)	Persistent segment tree (advanced)

14. Complexity Analysis

Operation	Time	Space
Segment Tree — Build	$O(n)$	$O(4n)$
Segment Tree — Query	$O(\log n)$	-
Segment Tree — Update	$O(\log n)$	-
Segment Tree — Lazy range	$O(\log n)$	$O(4n)$ extra for lazy array

Operation	Time	Space
update		
Fenwick Tree — Build	$O(n \log n)$ or $O(n)$	$O(n)$
Fenwick Tree — Query	$O(\log n)$	-
Fenwick Tree — Update	$O(\log n)$	-
2D Fenwick Tree — Query/Update	$O(\log^2 n)$	$O(n \times m)$

15. Common Mistakes

Mistake 1: Under-allocating the segment tree array

Using `new int[2*n]` instead of `new int[4*n]` works for perfect binary trees but breaks for other sizes. Always use $4n$ — the extra memory is negligible.

Mistake 2: Wrong Fenwick tree indexing (0 vs 1)

```
// Fenwick trees are 1-INDEXED. Index 0 is unused.
// If your input is 0-indexed, ALWAYS convert:

bit.update(i + 1, val);
bit.query(i + 1);
```

Mistake 3: Confusing set-to-value with add-delta

Fenwick trees only support ADD operations. To "set index i to value X ," compute $\text{delta} = X - \text{current}[i]$ and add that. Forgetting to track current values breaks the abstraction.

Mistake 4: Using Fenwick for non-invertible operations

Range MIN and MAX cannot use Fenwick trees because you can't "undo" a min operation. When a range shrinks, you'd have no way to recompute the new min. Use segment trees for min/max.

Mistake 5: Forgetting the identity element

```
// Sum tree: identity is 0
if (r < lo || hi < l) return 0;

// Min tree: identity is Integer.MAX_VALUE
if (r < lo || hi < l) return Integer.MAX_VALUE;

// GCD tree: identity is 0 (gcd(0, x) = x)
```

Mistake 6: Integer overflow

Sum queries over 10^5 elements each of size 10^9 overflow int. Use long for both tree and query results when values can be large.

Mistake 7: Lazy propagation errors

Forgetting to push before recursing into children. Not multiplying the lazy value by segment length. Forgetting to zero out lazy after propagation. Lazy propagation is delicate — always test with 3-4 mixed range updates and queries.

16. Best Practices

- Default to Fenwick tree for range SUM problems — smaller, faster, easier to write
- Use segment tree for range MIN, MAX, GCD, or any non-invertible operation
- Allocate segment tree as $4n$ for safety
- Use 1-indexed access on Fenwick trees — convert 0-indexed inputs at the boundary
- For large or sparse values, use coordinate compression to keep the tree small
- Use long instead of int for sum problems where values might overflow
- Only reach for lazy propagation when RANGE updates are needed — otherwise skip the complexity
- Draw a small tree on paper before coding — helps catch off-by-one and index errors

17. Quick Reference

Concept	Meaning
Segment Tree	Binary tree with array-aggregated ranges
Fenwick Tree (BIT)	Compact alternative for INVERTIBLE operations
Combine step	Change (sum, min, max, gcd) to change the tree's purpose
Identity element	0 for sum, MAX_VALUE for min, MIN_VALUE for max
Tree size	$4n$ for segment tree, $n+1$ for Fenwick
Query time	$O(\log n)$
Update time	$O(\log n)$
i & $-i$	Lowest set bit — Fenwick's navigation trick
Lazy propagation	For $O(\log n)$ RANGE updates
Coordinate compression	Map big values to small ranks — companion trick

Concept	Meaning
Delta trick	"Set to X" = add (X - current) in Fenwick

18. Practice Problems

Try in order — foundational range sum first, then min/max, then advanced.

Foundational

4. Range Sum Query — Mutable (LeetCode 307).
5. Range Sum Query 2D — Mutable (LeetCode 308).
6. Implement a Fenwick tree from scratch and test with range sums.
7. Implement a segment tree from scratch (sum, min, max variants).

Fenwick Tree Applications

8. Count of Smaller Numbers After Self (LeetCode 315).
9. Count of Range Sum (LeetCode 327).
10. Reverse Pairs (LeetCode 493).
11. Number of Inversions in an array.
12. Create Sorted Array through Instructions (LeetCode 1649).

Segment Tree Applications

13. Range Minimum Query on a static array (Sparse table variant also works).
14. Longest Increasing Subsequence with segment tree — $O(n \log n)$ variant.
15. Falling Squares (LeetCode 699).
16. The Skyline Problem (LeetCode 218) — with segment tree.
17. Rectangle Area II (LeetCode 850) — segment tree + coordinate compression.

Lazy Propagation

18. Range update + range query — implement lazy propagation.
19. Corporate Flight Bookings (LeetCode 1109) — compare difference array vs segment tree.
20. Range Addition II (LeetCode 598).

19. Final Takeaway

Segment trees and Fenwick trees are the natural next step after prefix sums. They solve the exact problem prefix sums couldn't: FREQUENT range queries mixed with FREQUENT updates. Every operation drops to $O(\log n)$ — a huge win over the $O(n)$ rebuild that prefix sums would need.

For interviews, master four things: the segment tree template for range SUM, swap it to range MIN/MAX by changing one line, the Fenwick tree template for range SUM (simpler, faster), and the coordinate compression companion trick. Lazy propagation is a bonus — worth understanding but rarely needed in interviews.

Key Takeaway: Segment trees give $O(\log n)$ range queries + point updates for any associative operation. Fenwick trees are the compact alternative for invertible ops (sums, XOR). Use Fenwick when the operation is SUM; segment tree when it's MIN/MAX/GCD. Both dwarf prefix sums as soon as updates are involved. Combine with coordinate compression for large value spaces. Lazy propagation gives $O(\log n)$ range updates — powerful but delicate.

20. What's Next?

With segment/Fenwick trees covered, the remaining specialized DSA topics are:

- KMP / Rabin-Karp — string pattern matching in $O(n + m)$
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points, bridges
- Number Theory basics — modular arithmetic, primes, GCD tricks, combinatorics
- Randomized Algorithms — quickselect, reservoir sampling
- Suffix Arrays / Automata — for advanced string problems
- Minimum Spanning Tree — Kruskal (already done in Ch 37) + Prim's
- System Design fundamentals — the natural big step for senior interviews